Environment Setup (Code-Along)

(https://github.com/learn-co-curriculum/python-p3-environment-setup) (https://github.com/learn-co-curriculum/python-p3-environment-setup/issues/new)

Learning Goals

- Use pyenv to use different Python versions
- Set up virtual environment using pipenv

Key Vocab

- **Module**: a file containing Python definitions and statements. A module's functions, classes, and global variables can be accessed by other modules.
- **Package**: a collection of modules that can be accessed as a group using the package name.
- `import` : the Python keyword used to access data from other packages and modules inside of the current module.
- **PyPI**: the **Py**thon **P**ackage **I**ndex. A repository of Python packages that can be downloaded and made available to your application.
- `pip` : the command line tool used to download packages from PyPI. `pip` is installed on your computer automatically when you download Python.
- **Virtual Environment**: a collection of modules, packages, and scripts that can be activated or deactivated at any time.
- **Pipenv**: a virtual environment tool that uses `pip` to manage the modules, packages, and scripts that you intend to use in your application.

Introduction

In this lesson we will talk about environment tools like pyenv and pipenv. These tools allows us to separate different Python environments for all kinds of use cases. This is useful when we need an isolated environment with libraries and specific versions of Python.

You should execute the commands shown in this lesson to get practice working with pyenv and pipenv.

Pyenv

In an earlier lesson, you installed pyenv and Python 3.8.13. (If needed, you can use the following instructions to install pyenv for your operating system.)

pyenv install instructions (https://github.com/pyenv/pyenv#installation)

Run the following command to confirm pyenv was installed for version 3.8.13:

```
$ pyenv versions
  system
* 3.8.13 (set by /Users/username/.pyenv/version)
  ...
```

Confirm your current global Python version is 3.8.13:

```
$ python3 --version
Python 3.8.13
```

Take a look at the file `program.py`, which simply prints the current Python version:

```
import sys
print("Python version")
print(sys.version)
```

You can also run this program to confirm your Python version is 3.8.13:

```
$ python program.py
Python version
```

```
3.8.13
```

We can see the different versions of Python available to install using the following command

```
$ pyenv install -l
Available versions:
  2.1.3
  2.2.3
  2.3.7
  ...
  ...
```

Lets install a new version of Python (3.11). We can do this using the following command

```
$ pyenv install 3.11
```

Installing a new version of Python does not however change the current global version. We can confirm this by checking the Python version:

```
$ python3 --version
Python 3.8.13
```

Use `pyenv` to set the global Python version to the new version 3.11:

```
$ pyenv global 3.11
```

Confirm the current global Python version (full version 3.11.x may differ):

```
$ python3 --version
Python 3.11.4
```

Now if we run a Python program, we'll see it is executed using Python version 3.11.x. For example, let's run `program.py` :

```
$ python program.py
Python version
3.11.4
```

Let's set the global Python version back to 3.8.13:

```
$ pyenv global 3.8.13
```

Confirm the global Python version:

```
$ python3 --version
Python 3.8.13
```

Confirm our Python program `program.py` runs in this version:

```
$ python program.py
Python version
3.8.13
```

Pipenv

Pyenv allows us to manage the version of Python we are working with.

What if we want to manage dependencies as well?

Pipenv automatically creates and manages a virtualenv for your projects, it also adds/removes packages from your Pipfile as you install/uninstall packages. Pipenv also let's you run programs with a particular version of Python.

To install pipenv follow the instructions here **pipenv install (https://pipenv.pypa.io/en/latest/installation/)**

We can use pipenv to create a virtual environment to run Python programs with version 3.11:

```
pipenv --python 3.11
```

In the Python flag we can define which version of `--python` we want to use.

You will notice a `Pipfile` and a `Pipfile.lock` have been added to the directory.

The `Pipfile` describes the dependencies we have installed and the Python version.

```
[[source]]
url = "https://pypi.org/simple"
verify_ssl = true
name = "pypi"

[packages]

[dev-packages]

[requires]
python_version = "3.11"
python_full_version = "3.11.4"
```

The `Pipfile.lock` describes all the dependencies our dependencies rely on. The lock file gives us the ability to produce a deterministic and reproducible project environment.

We can now install a library into the virtual environment. Lets use the `requests` library as an example

```
$ pipenv install requests
```

if we want to install a specific version of the requests library we can pin the version in the command

```
$ pipenv install requests==2.28.1
```

Pipenv has created a virtual environment for our project. We can see the location of this virtual environment by using the command (your location will differ):

```
$ pipenv --venv
Successfully created virtual environment!
Virtualenv location: /Users/username/python-p3-environment-setup/.venv
```

Pipenv allows us to install dev dependencies. We can do so by adding the `--dev` argument.

```
pipenv install pytest --dev
```

Pytest will be added to the Pipfile as a dev dependency

Dev dependencies are modules which are not needed in production but they help us develop and test the code.

The command `pipenv --version 3.11` created the `Pipfile` and `Pipfile.lock` files that define a virtual environment. However, the environment is not yet activated, and the current shell is still running Python 3.8.13. We can confirm this by:

```
$ python3 --version
Python 3.8.13
```

We can also run `program.py` to print the current Python version:

```
$ python program.py
Python version
3.8.13
```

How do we get into our virtual environment to run programs with version 3.11 and the various imported modules?

Use the following command to activate the virtual environment and run commands inside of it:

```
pipenv shell
```

Now we are in the virtual environment running Python 3.11, and have access to the dependencies we defined in the Pipfile. If we run the following commands in the virtual environment we will see that we can import the `requests` module we installed previously using `pipenv install requests`.

```
$ python
>>> import requests
>>> exit()
```

If we run our Python program `program.py` in the virtual environment, we should see we are running 3.11.x:

```
$ python program.py
Python version
3.11.4
```

If you ever need to remove a virtual environment you can use the `pipenv --rm` command.

```
$ pipenv --rm
Removing virtualenv (/Users/username/python-p3-environment-setup/.venv)...
```

This should return us back to our global Python version of 3.8.13, which we can confirm in a few different ways:

```
$ python3 --version
Python 3.8.13
```

```
$ python program.py
Python version
3.8.13
```

If you type `pipenv shell` (but don't!), you are activating the virtual environment in a new shell and every command you type is executed within that environment. Sometimes you might want to run a single command in a virtual environment. We can do with by running `pipenv install` then `pipenv run <command>` to execute a particular command within the virtual environment.

Run `pipenv install` again, but **don't** activate the environment in a new shell. You'll see the global version of Python for the current shell is still 3.8.13:

```
$ pipenv install
$ python program.py
Python version
3.8.13
```

We can run `program.py` in the virtual environment defined in `Pipfile` (i.e. Python 3.11 and installed modules) as shown:

```
$ pipenv run python program.py
Python version
3.11.4
```

Since we did not activate the virtual environment in a new shell, the environment is no longer active after the program completes, and the current shell is still using 3.8.13:

```
$ python program.py
Python version
3.8.13
```

Since we've set the python version for this program to 3.11, the program will run using python 3.11, even though the global version is 3.8.13. As long as we have a specific python version installed on our computer, we can use it locally in specific programs without having to reset our global python version.

Conclusion

Confirm you are no longer in a virtual environment (you might see an error message if you already removed the virtual environment).

```
$ pipenv --rm
```

Confirm you are back using global Python version 3.8.13:

```
$ python3 --version
Python 3.8.13
```

Virtual environments allow us to have a deterministic and predictable runtime for our Python projects. We can define specific versions for Python and the dependencies we need. We can easily add new virtual environments for multiple projects we have on our machine.

Resources

- **Python 3 Documentation (https://docs.python.org/3/)**
- **Pyenv (https://github.com/pyenv/pyenv/)**
- **Pipenv (https://pipenv.pypa.io/en/latest/)**